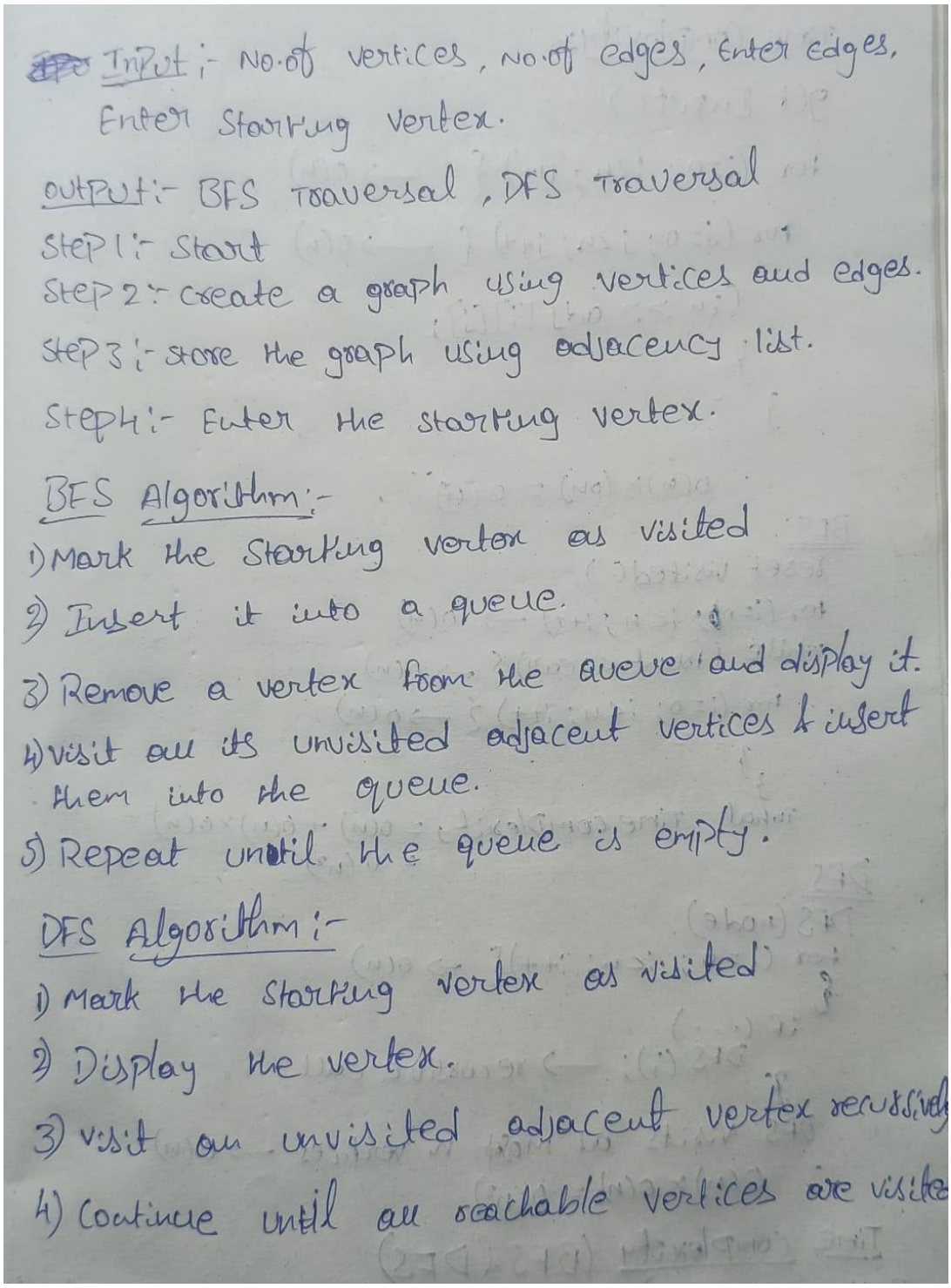
PRACTICAL-08

AIM : Implementation of Graph and Searching (DFS and BFS).

ALGORITHM :



Input :- No. of vertices, No. of edges, Enter edges, Enter Starting vertex.

Output :- BFS Traversal, DFS Traversal

Step 1 :- Start

Step 2 :- Create a graph using vertices and edges.

Step 3 :- Store the graph using adjacency list.

Step 4 :- Enter the starting vertex.

BFS Algorithm :-

1) Mark the starting vertex as visited
2) Insert it into a queue.
3) Remove a vertex from the queue and display it.
4) Visit all its unvisited adjacent vertices & insert them into the queue.
5) Repeat until the queue is empty.

DFS Algorithm :-

1) Mark the starting vertex as visited
2) Display the vertex.
3) Visit an unvisited adjacent vertex recursively
4) Continue until all reachable vertices are visited

PROGRAM :

```
#include <iostream>
using namespace std;
class Graph
{
public:
int adj[10][10];
int visited[10];
int n;
void getInput()
{
int edges;
cout << "Enter number of vertices: "; cin >> n;
for(int i = 0; i < n; i++)
{
for(int j = 0; j < n; j++)

{
adj[i][j] = 0;
}
}
cout << "Enter number of edges: ";
cin >> edges;
cout << "Enter edges:\n";
for(int i = 0; i < edges; i++)
{
int u, v;
cin >> u >> v;
adj[u][v] = 1;
adj[v][u] = 1;
}
}
void displayGraph() {
cout << "\nAdjacency Matrix:\n";
for(int i = 0; i < n; i++){
for(int j = 0; j < n; j++)
{
cout << adj[i][j] << " ";
}
cout << endl;
}
}
void DFS(int start)
```

```
{
visited[start] = 1;
cout << start << " ";
for(int i = 0; i < n; i++){
if(adj[start][i] == 1 && visited[i] == 0) {
DFS(i);
}
}
}
void BFS(int start)
{
int queue[10];
int front = 0;
int rear = 0;
for(int i = 0; i < n; i++) {
visited[i] = 0;
}

visited[start] = 1;

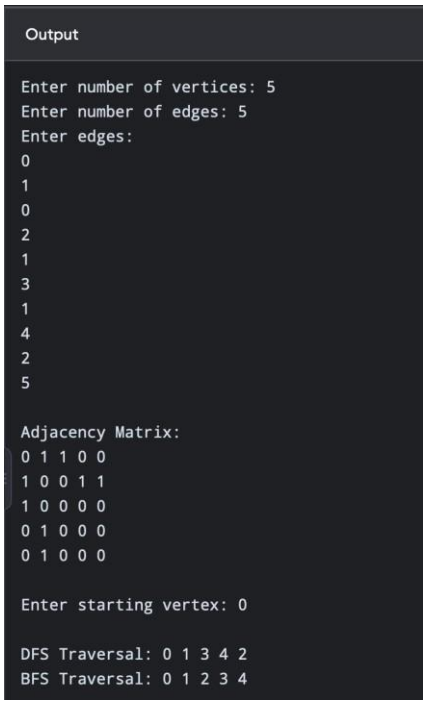
queue[rear] = start;

rear++;

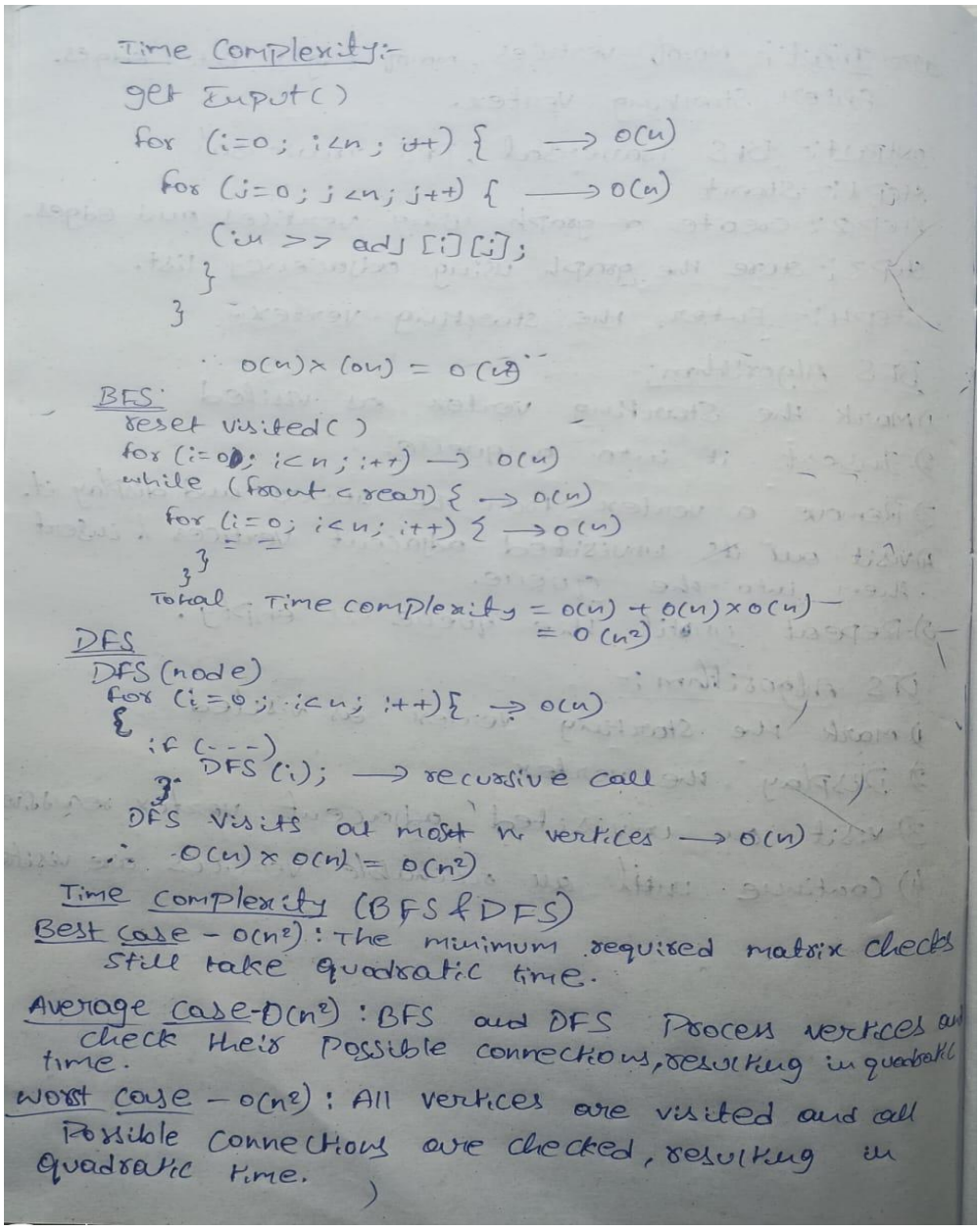
while(front < rear)
{
int vertex = queue[front];
front++;
cout << vertex << " ";
for(int i = 0; i < n; i++)
{
if(adj[vertex][i] == 1 && visited[i] == 0)
{
visited[i] = 1;
queue[rear] = i;
rear++; }
}
}
}
};
int main()
{
Graph g;
int start;
```

```
g.getInput();
g.displayGraph();
cout << "\nEnter starting vertex: ";
cin >> start;
for(int i = 0; i < g.n; i++)
{
g.visited[i] = 0;
}
cout << "\nDFS Traversal: ";
g.DFS(start);
cout << "\nBFS Traversal: ";
g.BFS(start);
cout << endl; return 0;
}
```

OUTPUT :



TIME COMPLEXITY :



Time Complexity:-
get Input()
for (i=0; i<n; i++) { → O(n)
for (j=0; j<n; j++) { → O(n)
(iu >> adj [i] [j];
}
}
O(n) × (on) = O(n²)
BFS
reset visited()
for (i=0; i<n; i++) → O(n)
while (front < rear) { → O(n)
for (i=0; i<n; i++) { → O(n)
}
}
Total Time complexity = O(n) + O(n) × O(n) –
= O(n²)
DFS
DFS (node)
for (i=0; i<n; i++) { → O(n)
{
if (---)
DFS (i); → recursive call
}
DFS visits at most n vertices → O(n)
∴ O(n) × O(n) = O(n²)
Time Complexity (BFS & DFS)
Best case – O(n²): The minimum required matrix checks still take quadratic time.
Average case-O(n²): BFS and DFS Process vertices and check their possible connections, resulting in quadratic time.
Worst case – O(n²): All vertices are visited and all Possible Connections are checked, resulting in Quadratic time.

CONCLUSION:

I implemented a Graph data structure and performed graph traversal using Depth First Search (DFS) and Breadth First Search (BFS). I represented the graph using an adjacency list and used a visited array to keep track of the vertices that had already been explored. In DFS, I explored each adjacent vertex deeply before backtracking, while in BFS, I used a queue to visit the vertices level by level. I tested both algorithms with different graph inputs and verified their traversal orders. Through this implementation, I understood the practical difference between DFS and BFS and learned how both algorithms can efficiently traverse a graph with a time complexity of O(V + E).